

Dependency Injection in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

Dependency Injection in ASP.NET Core Web API

Building modern applications requires writing clean, testable, and maintainable code. One of the most common challenges developers face is **managing dependencies**, which is the way one class relies on another to perform its work. Without a proper mechanism, dependencies can quickly lead to tightly coupled, rigid, and hard-to-test systems.

In this post, we will explore Dependency Injection in ASP.NET Core Web API in detail, starting from its necessity, understanding its design pattern, exploring service registration methods, implementing real-world examples, and finally, highlighting its advantages. By the end, you will have a strong understanding of how to use DI effectively in your ASP.NET Core applications.

Understanding the Need for Dependency Injection in ASP.NET Core

In **Traditional Applications**, classes often create and manage their own dependencies using the **new keyword**. This leads to **Tight Coupling** between classes, making the code difficult to maintain, extend, and test. So, when components create their own dependencies (tight coupling), we get several pain points:

- **Tight Coupling:** Changing a concrete implementation (e.g., swapping an in-memory store for a database) triggers edits across the codebase.
- **Difficult Testing:** Unit testing becomes more challenging as it is difficult to easily substitute dependencies with mocks.
- **Code Duplication:** Instantiating the same services in multiple places leads to redundancy.
- **Violation of SRP and OCP:** Your classes end up doing too much, and changes require touching multiple parts of the system.

ASP.NET Core solves this by introducing **Dependency Injection** at the **Framework Level**, allowing you to **Define Dependencies Externally** and **inject them Wherever Needed**. This ensures:

- Loose Coupling between classes.
- Centralized Dependency Management.
- Easier Testing using mock services.

Real-time Analogy:

Without Dependency Injection (Tight Coupling)

Imagine you own a **Restaurant**, and every time you need vegetables, **you go to the farm yourself, pick them, wash them, and then use them for cooking.**

- You are doing **Everything Yourself**.
- If the farm changes location, you have to adjust your routine.
- If you want organic vegetables instead, you must change your whole process.
- You are **Tightly Dependent** on the farm.

With Dependency Injection (Loose Coupling)

Now imagine instead of going to the farm, you have a **Supplier** who delivers vegetables directly to your restaurant.

- You don't care where the vegetables come from (farm A, farm B, organic supplier, wholesale market).
- You just **received the Vegetables** and focus on cooking.
- If you want to change suppliers, you don't change your cooking process; you just change who delivers.
- You are **Loosely Dependent** and much more flexible.

Key Idea

- **Without DI:** You (the restaurant) are responsible for creating and managing your own dependencies (vegetables).
- **With DI:** Someone else (the supplier/DI container) provides what you need, and you use it.

Product Management Example without Dependency Injection

When learning Dependency Injection (DI) in ASP.NET Core, we first need to understand the life of a codebase without DI. This helps us clearly identify the pain points, challenges, and drawbacks, and understand why DI is so powerful.

In this walkthrough, first, we will create a **Product Management Web API** using the traditional approach where controllers **Directly Create Their Own Dependencies instead of relying on DI**. Once we have this example without DI, we will discuss the drawbacks and how DI solves them.

Our example consists of three parts:

1. **Model (Product.cs)** – Defines the Product entity used across the system.
2. **Service (ProductService.cs)** – Encapsulates the product data store and related business logic, using an in-memory list.
3. **Controller (ProductsController.cs)** – Handles HTTP requests and returns responses to the client by directly calling the service.

Before starting, create a new **ASP.NET Core Web API Project** and name it **ProductManagement**. Inside the project root, add two new folders:

- Models – to store the entity classes.
- Services – to store business logic classes.

Creating Product Model

A model is simply a **POCO (Plain Old CLR Object)** class that defines the data structure. In our case, the Product model describes what a product looks like in the system. This model is used:

- By the Service Layer (to manage products in memory).
- By the Controller (to return structured responses to clients). You can also use DTOs.

So, create a class file named **Product.cs** within the **Models** folder, and copy-paste the following code. This class acts as the **contract** for product data in our system.

```
namespace ProductManagement.Models
{
    // Represents a product item exposed by the API.
    public sealed class Product
    {
        public int Id { get; set; } // Unique Identifier
        public string Name { get; set; } = null!; // Product Name
        public decimal Price { get; set; } // Product Price
        public string? Description { get; set; } //Optional Product Description
        public string? Category { get; set; } //Optional Category Label (e.g.,
        Electronics).
    }
}
```

Creating Product Service

The **Service** class encapsulates all **Business Logic and Data Handling**. In our example, it manages an **In-Memory List of Products**, but in real-time, you need to communicate with a database. The controller will call this service whenever it needs to interact with the product data.

So, create a class file named **ProductService.cs** within the **Services** folder and then copy-paste the following code. This class **centralizes business logic**, but without DI, it must be instantiated manually in the controller.

```
using ProductManagement.Models;
namespace ProductManagement.Services
{
    // Very simple in-memory product store.
```

```

// This class is NOT registered with DI in this example.
// The controller directly creates an instance of this service.
public sealed class ProductService
{
    // The in-memory "table" with hardcoded items.
    private readonly List<Product> _store =
    [
        new Product
        {
            Id = 1,
            Name = "Notebook",
            Price = 199.00m,
            Description = "A ruled paper notebook (100 pages).",
            Category = "Stationery"
        },
        new Product
        {
            Id = 2,
            Name = "Wireless Mouse",
            Price = 499.00m,
            Description = "2.4 GHz wireless mouse with ergonomic design.",
            Category = "Electronics"
        }
    ];

    // Returns all products ordered by Id.
    public IEnumerable<Product> GetAll()
    {
        return _store.OrderBy(p => p.Id).ToList();
    }

    // Returns a single product by Id, or null if not found.
    public Product? GetById(int id)
    {
        return _store.FirstOrDefault(p => p.Id == id);
    }

    // Creates a new product with a generated Id.
    public Product Create(Product input)
    {
        if (string.IsNullOrWhiteSpace(input.Name))
            throw new ArgumentException("Product name is required.",
nameof(input.Name));

        if (input.Price < 0)
            throw new ArgumentException("Price cannot be negative.",
nameof(input.Price));
    }
}

```

```

        // Generate new Id (max existing Id + 1)
        var newId = _store.Any() ? _store.Max(p => p.Id) + 1 : 1;

        var product = new Product
        {
            Id = newId,
            Name = input.Name.Trim(),
            Price = input.Price,
            Category = string.IsNullOrWhiteSpace(input.Category) ? null :
input.Category.Trim(),
            Description = string.IsNullOrWhiteSpace(input.Description) ? null
: input.Description.Trim()
        };

        _store.Add(product);
        return product;
    }

    // Updates an existing product; returns false if the Id doesn't exist.
    // Only provided fields are applied; others are kept as-is.
    public bool Update(int id, Product input)
    {
        var existing = _store.FirstOrDefault(p => p.Id == id);
        if (existing == null) return false;

        existing.Name = string.IsNullOrWhiteSpace(input.Name) ? existing.Name
: input.Name.Trim();
        existing.Price = input.Price < 0 ? existing.Price : input.Price;
        existing.Category = string.IsNullOrWhiteSpace(input.Category) ?
existing.Category : input.Category.Trim();
        existing.Description = string.IsNullOrWhiteSpace(input.Description) ?
existing.Description : input.Description.Trim();

        return true;
    }

    // Deletes a product; returns false if the Id doesn't exist.
    public bool Delete(int id)
    {
        var product = _store.FirstOrDefault(p => p.Id == id);
        if (product == null) return false;

        _store.Remove(product);
        return true;
    }
}

```

```
}
```

Creating Products Controller

The **Controller** is the entry point for HTTP requests. In our example, it will expose REST API endpoints to perform CRUD operations on products.

So, create an empty API controller named **ProductsController** within the **Controllers** folder and then copy-paste the following code. Notice that the controller **Directly Instantiates ProductService**. This is easy for small demos, but creates big issues in larger apps.

```
using Microsoft.AspNetCore.Mvc;
using ProductManagement.Models;
using ProductManagement.Services;
namespace ProductManagement.Controllers
{
    // Web API controller for managing products.
    // This version demonstrates NO dependency injection:
    // - The controller owns a ProductService instance.
    // - In real-world apps, prefer DI for testability and flexibility.
    [ApiController]
    [Route("api/[controller]")]
    public sealed class ProductsController : ControllerBase
    {
        // Option A (per-controller instance):
        // private readonly ProductService _service = new();

        // Option B (shared app-wide instance): use static (persists for process
lifetime)
        // Using static here so the in-memory data doesn't reset every request.
        private static readonly ProductService _service = new();

        // GET /api/products -> returns all products.
        [HttpGet]
        public ActionResult<IEnumerable<Product>> GetAll()
        {
            // No DI: we call the service field directly
            var products = _service.GetAll();
            return Ok(products);
        }

        //GET /api/products/{id} -> returns product by id.
        [HttpGet("{id:int}")]
        public ActionResult<Product> GetById(int id)
        {
            var product = _service.GetById(id);
        }
    }
}
```

```

        return product is null ? NotFound() : Ok(product);
    }

    // POST /api/products -> creates a new product.
    [HttpPost]
    public ActionResult<Product> Create([FromBody] Product input)
    {
        try
        {
            var created = _service.Create(input);
            // Returns 201 with Location header pointing to GET by id
            return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
        }
        catch (Exception ex)
        {
            // Bad request when validation fails at the service layer
            return BadRequest(new { error = ex.Message });
        }
    }

    //PUT /api/products/{id} -> updates an existing product.
    [HttpPut("{id:int}")]
    public IActionResult Update(int id, [FromBody] Product input)
    {
        var ok = _service.Update(id, input);
        return ok ? NoContent() : NotFound();
    }

    // DELETE /api/products/{id} -> deletes an existing product.
    [HttpDelete("{id:int}")]
    public IActionResult Delete(int id)
    {
        var ok = _service.Delete(id);
        return ok ? NoContent() : NotFound();
    }
}
}

```

Problems in the Non-DI Implementation

This non-DI approach **works for demos** but introduces real-world issues as your app grows. Some of the problems are as follows:

1. **Tight Coupling:** The controller directly depends on the concrete **ProductService**. If you later need to switch to an **EF Core Repository** or an **External API**, you must **edit the Controller Code**. With DI, you can swap

implementations in one place,

2. **Hard to Test:** Because the controller owns the service, you **cannot inject a Fake/Mock** service for unit tests. You are forced to hit the real in-memory store, which makes tests awkward and less isolated.
3. **Maintenance Overhead:** If ProductService gains constructor parameters (e.g., options, logger, client), **Every Place** that creates it must change. With DI, the container handles construction and lifetime management.
4. **Violates SRP (Single Responsibility Principle):** The controller is now doing **two jobs: handling HTTP Requests** and **creating/Managing Dependencies**. SRP urges separation; controllers should handle requests, not build infrastructure.
5. **No Lifetime Control:** You can't express whether a service should be **Transient** (new per use), **Scoped** (per request), or **Singleton** (app lifetime).

What is the Dependency Injection (DI) Design Pattern?

Dependency Injection is a **Design Pattern** that **Removes Dependency Creation** from a class and instead provides (injects) them externally.

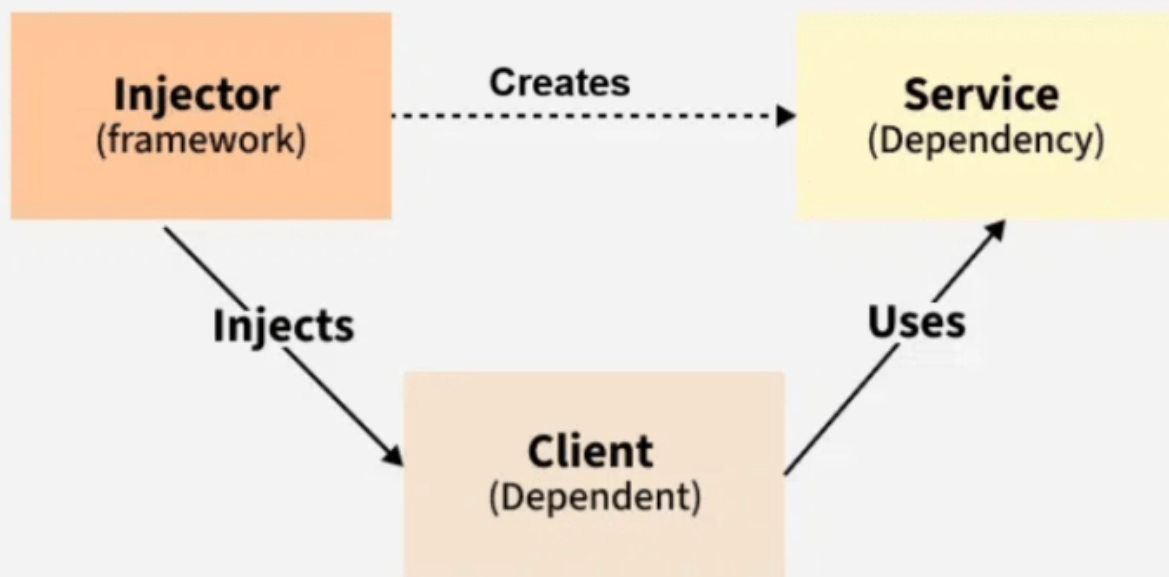
- **Without DI:** Class A creates an instance of Class B.
- **With DI:** Class A receives an instance of Class B from outside (Container).

ASP.NET Core provides a **Built-In DI Container** to register and resolve dependencies automatically at runtime.

Components involved in the Dependency Injection (DI) Design Pattern

There are four components involved in the Dependency Injection Design Pattern. For a better understanding, please have a look at the following diagram. Here, Service represents two things: the Interface and its implementation class.

Dependency Injection(DI) Design Pattern



Abstraction (Interface / Contract)

An abstraction is like a **contract** that defines what the service can do, but not how it does it. In C#, this is usually an **interface**. The client depends only on this abstraction, not on the actual service implementation. This way, the client doesn't care *which* service it gets, as long as it follows the contract.

Key Points:

- Defines **what needs to be done**, not how.
- Usually written as an **interface** in code (e.g., `IProductService`).
- Makes the system **flexible** — you can easily swap implementations.
- Keeps the **client (user)** free from knowing service details.

Service (Dependency / Implementation)

A service is the class that actually **does the work**. It provides some functionality that other parts of the application need. It contains the logic that your application needs, like sending an email, saving data to a database, or processing a payment. Without the service, your program cannot complete its tasks.

Key Points:

- It is the **real worker** of your system.
- Contains the actual **business logic**.
- It is the **dependency** that other classes need.

- Examples: EmailService, ProductRepository, PaymentService.
- Can have **multiple versions** (e.g., EmailService using Gmail or Outlook).

Client (Consumer / Dependent Class)

A client is the class that **needs the service** to perform its job. It doesn't do the actual work itself but depends on a service to get things done. The client depends on the service but does not care about how it is created. For example, an OrderController needs an IEmailService to send order confirmation emails.

Key Points:

- It is the **user of the service**.
- Relies on an abstraction (interface), not on the concrete service.
- It only **uses** the service; it does not **create** it.
- Example: A controller using IProductService for CRUD operations.
- Becomes **simpler and cleaner** because it doesn't create dependencies on its own.

Injector (Dependency Injection Container)

The injector is like the **manager** who provides the right worker (service) to the client. In ASP.NET Core, this is the **built-in DI container**. It creates service objects, gives them to the classes that need them, and manages their lifetime (i.e., how long they live).

Key Points:

- Also called the **DI container**.
- **Registers** services with their matching interfaces.
- **Resolves** dependencies when a client needs them.
- **Injects** the correct implementation at runtime.
- Manages **lifetimes**: Transient (new every time), Scoped (per request), Singleton (once for the whole app).

Types of Services in ASP.NET Core: Custom vs Built-in Services

They are mainly of two types:

Built-in Services:

ASP.NET Core provides many built-in services, such as:

- **Logging** (ILogger<T>),
- **Configuration** (IConfiguration),

- **Options** (IOptions<T>, IOptionsMonitor<T>),
- **HTTP** (IHttpClientFactory),
- **Caching** (IMemoryCache, IDistributedCache),
- **Authentication/Authorization, Routing/MVC** services, etc.

Custom Services:

You can create your own services (e.g., IStudentRepository, EmailService) and register them in the DI container

- Email sending (IEmailService).
- Background job scheduler (IJobService).
- Notification manager (INotificationService).
- Third-party API integrations (e.g., SMS, payments).

How to Register a Service with the ASP.NET Core DI Container?

We need to register a service with the ASP.NET Core Dependency Injection Container within the Main method of the Program class. All registrations happen in the Program.cs (composition root) using builder.Services. ASP.NET Core provides three main lifetimes:

- **Singleton:** One instance throughout the application. Syntax: `builder.Services.AddSingleton<IProductRepository, ProductRepository>();`
- **Scoped:** One instance per HTTP request. Syntax: `builder.Services.AddScoped<IProductRepository, ProductRepository>();`
- **Transient:** New instance every time it's requested. Syntax: `builder.Services.AddTransient<IProductRepository, ProductRepository>();`

Product Management Example with Dependency Injection

In our earlier non-DI example, the controller **created its own instance** of ProductService. This led to:

- Tight coupling (controller knows exact implementation).
- Difficult testing (cannot be replaced with mocks).
- No lifecycle control (service always behaves like a static singleton).

By introducing **Dependency Injection**:

- The controller depends only on an **interface** (IProductService) instead of a concrete class.
- ASP.NET Core's built-in DI container will create and manage the service's lifetime.
- We gain flexibility: swap implementations (in-memory, database, mock) without touching the controller.

Define the Service Contract (Interface)

First, create an interface **IProductService.cs** inside the **Services** folder and copy-paste the following code. This interface defines the operations our service must support. Now the controller can depend on **IProductService** instead of directly on **ProductService**.

```
using ProductManagement.Models;
namespace ProductManagement.Services
{
    // Defines the contract for product operations
    public interface IProductService
    {
        IEnumerable<Product> GetAll();
        Product? GetById(int id);
        Product Create(Product input);
        bool Update(int id, Product input);
        bool Delete(int id);
    }
}
```

Update the Product Service to Implement the Interface

Modify **ProductService** to implement **IProductService**. So, please replace the **ProductService** with the following code:

```
using ProductManagement.Models;
namespace ProductManagement.Services
{
    // In-memory implementation of IProductService.
    // Uses a List<Product> with inline seed data.
    // Thread-safety note: we guard all access with a simple lock (_lock)
    // because we register this as a Singleton for demo persistence.
    public sealed class ProductService : IProductService
    {
        private readonly List<Product> _store =
        [
            new Product
            {
                Id = 1,
                Name = "Notebook",
                Price = 199.00m,
                Description = "A ruled paper notebook (100 pages).",
                Category = "Stationery"
            },
            new Product
            {

```

```

        Id = 2,
        Name = "Wireless Mouse",
        Price = 499.00m,
        Description = "2.4 GHz wireless mouse with ergonomic design.",
        Category = "Electronics"
    }
];

// Simple lock to protect _store when running as a Singleton
private readonly object _lock = new();

public IEnumerable<Product> GetAll()
{
    lock (_lock)
    {
        return _store.OrderBy(p => p.Id).ToList();
    }
}

public Product? GetById(int id)
{
    lock (_lock)
    {
        return _store.FirstOrDefault(p => p.Id == id);
    }
}

public Product Create(Product input)
{
    if (string.IsNullOrEmpty(input.Name))
        throw new ArgumentException("Product name is required.",
nameof(input.Name));

    if (input.Price < 0)
        throw new ArgumentException("Price cannot be negative.",
nameof(input.Price));

    lock (_lock)
    {
        // Generate new Id (max existing Id + 1)
        var newId = _store.Any() ? _store.Max(p => p.Id) + 1 : 1;

        var product = new Product
        {
            Id = newId,
            Name = input.Name.Trim(),
            Price = input.Price,

```

```

        Category = string.IsNullOrWhiteSpace(input.Category) ? null :
input.Category.Trim(),
        Description = string.IsNullOrWhiteSpace(input.Description) ?
null : input.Description.Trim()
    };

    _store.Add(product);
    return product;
}

}

public bool Update(int id, Product input)
{
    lock (_lock)
    {
        var existing = _store.FirstOrDefault(p => p.Id == id);
        if (existing == null) return false;

        // Only update provided fields
        existing.Name = string.IsNullOrWhiteSpace(input.Name) ?
existing.Name : input.Name.Trim();
        existing.Price = input.Price < 0 ? existing.Price : input.Price;
        existing.Category = string.IsNullOrWhiteSpace(input.Category) ?
existing.Category : input.Category.Trim();
        existing.Description =
string.IsNullOrWhiteSpace(input.Description) ? existing.Description :
input.Description.Trim();

        return true;
    }
}

public bool Delete(int id)
{
    lock (_lock)
    {
        var product = _store.FirstOrDefault(p => p.Id == id);
        if (product == null) return false;

        _store.Remove(product);
        return true;
    }
}

}
}
}

```

Why the lock? We will register this service as a **Singleton** (so data doesn't reset between requests). Singletons can be accessed by multiple requests concurrently; the lock keeps the in-memory list consistent in this demo.

Register the service with the DI container

Now we need to tell ASP.NET Core Framework to inject **ProductService** wherever **IProductService** is needed. Wire **Interface** → **Implementation** and choose a **Lifetime**. For an in-memory demo, **Singleton** is convenient so your data isn't reset with every request. So, please update your **Program.cs** class file as follows:

```
using ProductManagement.Services;
namespace ProductManagement
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers();
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            // DI mapping: interface -> concrete type
            // Singleton here so seeded data persists across the app lifetime.
            builder.Services.AddSingleton<IProductService, ProductService>();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();

            app.MapControllers();

            app.Run();
        }
    }
}
```

Choosing the lifetime

- **Singleton**: one instance for the whole app (great for in-memory demo caches; ensure thread-safety).
- **Scoped** (most common for EF Core): one instance per HTTP request.
- **Transient**: new instance every resolution (lightweight, stateless services).

Refactor the Controller to Use DI

Now, we need to modify the controller to accept **IProductService** through **Constructor Injection**. The framework will **Automatically Provide** the registered **IProductService** instance at runtime.

```
using Microsoft.AspNetCore.Mvc;
using ProductManagement.Models;
using ProductManagement.Services;

namespace ProductManagement.Controllers
{
    // Web API controller for managing products.
    // This version demonstrates dependency injection:
    [ApiController]
    [Route("api/[controller]")]
    public sealed class ProductsController : ControllerBase
    {
        private readonly IProductService _service;

        // Constructor Injection: ASP.NET Core provides an implementation
        // automatically
        public ProductsController(IProductService service)
        {
            _service = service;
        }

        // GET /api/products -> returns all products.
        [HttpGet]
        public ActionResult<IEnumerable<Product>> GetAll()
        {
            var products = _service.GetAll();
            return Ok(products);
        }

        // GET /api/products/{id} -> returns product by id.
        [HttpGet("{id:int}")]
        public ActionResult<Product> GetById(int id)
        {
            var product = _service.GetById(id);
            return product is null ? NotFound() : Ok(product);
        }
    }
}
```



```

    }

    // POST /api/products -> creates a new product.
    [HttpPost]
    public ActionResult<Product> Create([FromBody] Product input)
    {
        try
        {
            var created = _service.Create(input);
            // Returns 201 with Location header pointing to GET by id
            return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
        }
        catch (Exception ex)
        {
            // Bad request when validation fails at the service layer
            return BadRequest(new { error = ex.Message });
        }
    }

    //PUT /api/products/{id} -> updates an existing product.
    [HttpPut("{id:int}")]
    public IActionResult Update(int id, [FromBody] Product input)
    {
        var ok = _service.Update(id, input);
        return ok ? NoContent() : NotFound();
    }

    // DELETE /api/products/{id} -> deletes an existing product.
    [HttpDelete("{id:int}")]
    public IActionResult Delete(int id)
    {
        var ok = _service.Delete(id);
        return ok ? NoContent() : NotFound();
    }
}
}

```

Code Explanation:

- The controller **does not create ProductService directly**.
- Instead, it asks for an IProductService in its constructor.
- ASP.NET Core automatically injects the correct implementation.
- This makes the controller **loosely coupled, testable, and flexible**.

Different Ways of Accessing Dependencies in ASP.NET Core Web API

There are 3 different approaches that we can use to inject the dependency object into a class in ASP.NET Core.

- **Constructor Injection – The Recommended Approach**
- **Action Method Injection – For Occasional Dependencies**
- **Manual Service Resolution – Use Only When Necessary**

Constructor Injection (The Recommended Approach)

Constructor Injection is the most commonly used and recommended method of injecting dependencies. The DI container automatically creates and passes dependencies to the class constructor at runtime. This ensures your object is fully initialized and ready to perform its intended tasks.

Why it's preferred

- **Explicit contract:** Anyone reading your constructor can see exactly what the class needs.
- **Completeness:** The object can't be created in an invalid state (no "forgot to set property X" problems).
- **Easy to test:** In unit tests, you can pass in fakes/mocks directly.
- **Lifetime safety:** The container controls lifetimes and disposal.

Syntax:

```
[ApiController]
[Route("api/[controller]")]
1 reference
public sealed class ProductsController : ControllerBase
{
    // Constructor Injection
    private readonly IProductService _service;

    0 references
    public ProductsController(IProductService service)
    {
        _service = service;
    }

    // GET /api/products
    [HttpGet]
    0 references
    public ActionResult<IEnumerable<Product>> GetAll()
    {
        var products = _service.GetAll();
        return Ok(products);
    }
}
```

Real-Time Use Cases

- Business services like `IProductService`, `IOrderService`, `ILogger<T>`, or `DbContext`.
- When the service is needed by **multiple action methods** in the same controller.
- Example: A `ProductsController` needs `IProductService` for CRUD operations.

Action-Method Injection (For Occasional Dependencies)

If a service is needed **only inside a specific action**, **one** or **a few** endpoints (not the whole controller), injecting it into the controller constructor would be unnecessary. Instead, use `[FromServices]` in the method signature, i.e., inject it **at the action parameter** using `[FromServices]`.

Syntax:

Real-Time Use Cases

- Dependencies that are **used only in one or two endpoints**
- A service used for **special cases, such as** sending an email notification, generating a report, or exporting data.
- Example:
- A **one-off service** for generating PDF reports (IReportService) used in only one endpoint.
- A **temporary feature, such as ICaptchaValidator**, is used only in the registration API.

Manually Resolving Services (Use Only When Necessary)

You need to use `HttpContext.RequestServices.GetService<T>()` to manually resolve a service at runtime. This is useful in **middleware**, **filters**, or in situations where injection isn't possible.

Syntax:

Real-Time Use Cases

Rare cases where you **cannot use constructor or method injection**, for example:

1. **Background jobs** (like Hangfire jobs) where you don't control object creation.
2. **Middleware components** where constructor injection is not practical.
3. **Dynamic plugin loading**, where the service type may not be known until runtime.

Example to Understand the above Approaches:

Please modify the ProductsController as follows. The following code shows the use of all three approaches. Please read the comment lines for a better understanding:

```
using Microsoft.AspNetCore.Mvc;
using ProductManagement.Models;
using ProductManagement.Services;

namespace ProductManagement.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public sealed class ProductsController : ControllerBase
    {
        // Constructor Injection
        // This is the most common and recommended approach.
        // - The dependency is declared in the constructor.
        // - ASP.NET Core's DI container automatically supplies it when the
        controller is created.
        // - The injected instance (_service) is then available to all actions in
        this controller.
        private readonly IProductService _service;

        public ProductsController(IProductService service)
        {
            _service = service;
        }

        // GET /api/products
        // Uses: Constructor-injected _service
        // Why: We need IProductService in multiple actions (CRUD),
        //       so constructor injection avoids duplication.
        // When to use: Any dependency required across multiple actions → default
        approach.
        [HttpGet]
        public ActionResult<IEnumerable<Product>> GetAll()
        {
            var products = _service.GetAll();
            return Ok(products);
        }

        // Action Method Injection
        // In this approach, a dependency is injected directly into a single
        action method
        // using the [FromServices] attribute.
        // - This keeps the constructor clean if the dependency is rarely used.
        // - The service exists only in this method's scope.
```

```

// Real-world example: a reporting/export service needed only by one
endpoint.

// GET /api/products/{id}
[HttpGet("{id:int}")]
public ActionResult<Product> GetById(
    int id,
    [FromServices] IProductService productService // injected only for
this action
)
{
    var product = productService.GetById(id);
    return product is null ? NotFound() : Ok(product);
}

// Manual Resolution (Service Locator)
// This approach fetches a service manually from the built-in DI
container
// using HttpContext.RequestServices.
// - This is discouraged in normal development (harder to test and
maintain).
// - It is shown here ONLY to contrast with the other two techniques.
// - Sometimes useful in edge cases where constructor/action injection
isn't possible
// (like dynamic resolution in middleware or factory patterns).

// POST /api/products
[HttpPost]
public ActionResult<Product> Create([FromBody] Product input)
{
    // Manually resolve IProductService from the service provider
    var productService =
HttpContext.RequestServices.GetRequiredService<IProductService>();

    try
    {
        var created = productService.Create(input);

        // Returns HTTP 201 (Created) with a Location header
        // pointing to the newly created resource (GetById).
        return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
    }
    catch (Exception ex)
    {
        // If service-level validation fails, return HTTP 400 (Bad
Request).

```

```
        return BadRequest(new { error = ex.Message });  
    }  
}  
}
```

Why Property Injection is Not Supported in ASP.NET Core

ASP.NET Core intentionally avoids **property injection** because it:

- Makes dependencies **implicit** (harder to discover, easier to forget to set).
- Breaks **constructor completeness**; objects can exist in an invalid state until properties are set.
- Complicates lifetime and nullability guarantees.

Advantages of Using Dependency Injection in ASP.NET Core

- **Loose Coupling**: Classes depend on abstractions (IService) instead of concrete implementations (Service). This reduces code dependencies and improves maintainability.
- **Testability**: Services can be easily mocked or stubbed during unit testing, allowing controllers or business logic to be tested in isolation.
- **Lifecycle Management**: DI container manages object creation, lifetimes (Transient, Scoped, Singleton), and disposal automatically. No need for manual memory management.
- **Maintainability & Scalability**: If you replace a service (e.g., in-memory → EF Core repository), no changes are required to the controller. Just update Program.cs.

Dependency Injection is not just a feature in ASP.NET Core. It is the **Foundation of Modern Application Development**. By delegating responsibility for dependency creation to the built-in IoC container, developers gain flexibility, maintainability, and testability.

In real-world projects, DI helps enforce clean architecture, decouples business logic from infrastructure, and provides a scalable way to manage services as applications grow. Whether through constructor injection, action injection, or service lifetimes, mastering DI ensures your ASP.NET Core Web API applications remain professional, robust, and future-proof.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server,

MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information